1. Graphs are a powerful tool to model many phenomena. The edges of a graph model pairwise relationships. It is natural to consider higher order relationships. Indeed *hypergraphs* provide one such modeling tool. A hypergraph $G = (V, \mathcal{E})$ consists of a finite set of nodes/vertices V and a finite set of hyper-edges $\mathcal{E}$. A hyperedge $e \in \mathcal{E}$ is simply a subset of nodes and the cardinality of the subset can be larger than two. An undirected graph is a hypergraph where each hyper-edge is of size exactly two.

   Here is an example: $V = \{1,2,3,4,5\}$, $\mathcal{E} = \{\{1,2\},\{2,3,4\},\{1,3,4,5\},\{2,5\}\}$. The representation size of a hypergraph is $|V| + \sum_{e \in \mathcal{E}} |e|$.

   An alternating sequence of nodes and edges $x_1, e_1, x_2, e_2, \ldots, e_{k-1}, x_k$ where $x_i \in V$ for $1 \le i \le k$ and $e_j \in \mathcal{E}$ for $1 \le j \le k-1$ is called a path from u to v if: (i) $x_1 = u$, $x_k = v$, and (ii) for all $1 \le j < k$, $x_j \in e_j$ and $x_{j+1} \in e_j$.

   Given a hypergraph $G = (V, \mathcal{E})$ and two nodes $u, v \in V$, we say that u is connected to v if there is a path from u to v. We say that a hypergraph is connected if each pair of nodes u, v in G are connected. Describe an algorithm that, given a hypergraph G, checks whether G is connected in linear time. In essence, describe a reduction of this problem to the standard graph connectivity problem.

Solution: The algorithm returns whether the hypergraph is connected.

Algorithm

(a) Construct G' as described:
- $V' = V \cup \mathcal{E}$
- $E' = \{(v, e) \mid v \in V, e \in \mathcal{E}, v \in e\}$

(b) Perform BFS on G', starting from an arbitrary vertex $u \in V$, counting the number of vertices in V that are visited.

(c) Return `true` if the number of visited vertices in V is equal to $|V|$, and `false` otherwise.

Claim For any vertices $u, v \in V$, there is a path from u to v in the hypergraph G if and only if there is a path from u to v in the graph G'.

Proof

($\Rightarrow$) Let $v_1, e_1, v_2, e_2, \ldots, e_{k-1}, v_k$ be a path from u to v in G, where $v_1 = u$, $v_k = v$, and for all $1 \le i < k$, we have $v_i \in e_i$ and $v_{i+1} \in e_i$. By the definition of E', both (v_i, e_i) and (e_i, v_{i+1}) are edges in G'. Therefore, the path

$$v_1, (v_1, e_1), e_1, (e_1, v_2), v_2, (v_2, e_2), e_2, \ldots, v_k$$

is a valid path from u to v in G'.

($\Leftarrow$) Note that G' is a bipartite graph with partitions V and $\mathcal{E}$. Any path in G' must alternate between vertices in V and $\mathcal{E}$. Let

$$v_1, (v_1, e_1), e_1, (e_1, v_2), v_2, (v_2, e_2), \ldots, v_k$$

be a path from u to v in G'. Then, by construction of E', we have $v_i \in e_i$ and $v_{i+1} \in e_i$ for all i. Hence, the path

$$v_1, e_1, v_2, e_2, \ldots, v_k$$

is a valid path in the hypergraph G.

Conclusion The number of vertices in V reachable from any vertex in V is the same in both G and G'. Therefore, the hypergraph G is connected if and only if all nodes in V are reachable from any other node in V via G'.

Run Time Let $|V'| = |V| + |\mathcal{E}|$ and $|E'| = \sum_{e \in \mathcal{E}} |e|$. Then:

- Constructing G' takes $O(|V'| + |E'|)$ time.
- BFS on G' also runs in $O(|V'| + |E'|)$ time.

Thus, the total running time is $O\left(|V| + |\mathcal{E}| + \sum_{e \in \mathcal{E}} |e|\right)$, which is linear in the size of the input. ■

2. BFS Tree Explosion with Degree Bound Let $G = (V, E)$ be a directed graph with n vertices, and suppose every vertex has out-degree at most m. Fix a start vertex s, and define a *BFS tree* as the parent-child tree returned by BFS starting at s, where the queue resolves neighbor order arbitrarily (i.e., not sorted). Two BFS trees are considered *distinct* if their parent-child structure differs. What is a tight upper bound on the maximum number of distinct BFS trees that can be produced from a single fixed start vertex s, as a function of n and m?

Solution: If each vertex has at most m out-neighbors, then the number of BFS trees starting from a fixed s is at most exponential in $n \log m$, i.e.,

$$T(n, m) \in O(m^n)$$

This is because every node can have up to m children, and each layer's queue permutation affects which node reaches the next layer first. ■

3. Assume we have a graph G with n vertices and m equal weight edges. We have two balls B_1, B_2 that will move along their designed path P_1, P_2, where P_1 has length N_1 and P_2 has length N_2. Both balls will start at $P_1[1]$ and $P_2[1]$, and can move only forward along their designed paths. To be more precise, we define a valid move for a ball is either stay in it's current node, or move to the next node on its path. Each ball makes exactly one legal move in each round. A sequence, specifying in each round a valid move for each robot, is a plan.

   The score of a plan is the maximum distance of the two balls from each other at any moment during the execution of the plan.

   Describe an algorithm that computes the minimum score plan for the two balls. Your algorithm should be as fast as possible.

Solution: We will build a look-up-table, denoted as $d_G(u,v)$, that will return the shortest distance between u and v. We can run BFS n times to build the value for the look-up-table. After that, the solution is a straightforward adaptation of the edit-distance DP. Let $l(i,j) = d_G(P_1[i], P_2[j])$, we can build the function minscore(i,j) that will return the minimum score plan for two balls if they are currently on position $P_1[i]$ and $P_2[j]$. And we have the following recurrence:
minscore(i,j) =

$$\begin{cases} l(N_1, N_2) & i = N_1 and j = N_2 \\ max(minscore(N_1, j+1), l(N_1, j)) & i = N_1 \\ max(minscore(i+1, N_2), l(i, N_2)) & j = N_2 \\ \alpha & otherwise \end{cases}$$

where $\alpha = max(min[minscore(i+1, j), minscore(i, j+1), minscore(i+1, j+1)], l(i, j))$. Since i and j are bounded by $O(N_1)$ and $O(N_2)$, there are at most $O(N_1 \cdot N_2)$ recursive function calls. We can use a 2-D array with size $O(N_1 \cdot N_2)$ for memoization. The filling order will be in decreasing i and decreasing j. And the value we are interested in is minscore(1,1). Since building the look-up-table l takes $O(nm)$ time, our total runtime will be $O(N_1 N_2 + nm)$. This problem is a variant of the discrete Fréchet distance problem. ■